Pruebas unitarias
Paso a paso basico

Proyecto Accesorios Chic - frontend-react

Para alguien que nunca ha hecho una prueba.

Idea en 3 lineas: una prueba es un archivito que comprueba que tu codigo devuelve lo esperado. Si lo rompes, el test se pone en rojo. Eso es todo. PowerShell es solo para comandos cortos (npm test). El codigo de pruebas se escribe en VS Code en archivos nuevos.

1. Que te falta exactamente

- Hoy tienes 0 archivos *.test.* y sin config de tests (verificado por glob). Sin script test en package.json.
- La evaluacion p.11 pide 5 tareas: 1) lista renderiza todo, 2) mensaje de error solo con error, 3) boton recibe etiqueta y onClick, 4) formulario cambia al escribir, 5) clic cambia estado.
- Las haremos con tus archivos reales: TarjetaProducto.tsx:7, Productos.tsx:32-63, Carrito.tsx:33-38 y 81-86.

POR QUE: sin estos 5 tests no hay documento de cobertura ni nota de testing, aunque la app funcione.

2. Preparacion (una sola vez)

DONDE: todo dentro de C:\Users\HP\Documents\GitHub\e-commerce\frontend-react

2.1 Instalar (PowerShell, una linea + Enter)

```
npm install -D vitest jsdom @testing-library/react @testing-library/jest-dom @testing-library/user-event
```

POR QUE Vitest y no Karma/Jasmine del PDF: Karma abre Chrome y falla en EC2 t2.micro y con Vite+TS. Vitest usa la misma sintaxis test/expect y si funciona con Vite.

2.2 Crear 3 archivos base (VS Code, no PowerShell)

Archivo 1: vitest.config.ts junto a vite.config.ts

```
import { defineConfig } from 'vitest/config';
import react from '@vitejs/plugin-react';
export default defineConfig({
  plugins: [react()],
  test: { environment: 'jsdom', setupFiles: ['./src/setupTests.ts'] }
});
```

Archivo 2: src/setupTests.ts

```
import '@testing-library/jest-dom';
```

Archivo 3: en package.json, en scripts, agregar:

```
"test": "vitest run"
```

Verificacion: corre npm test. Si dice No test files found esta bien: significa que la config carga.

3. Base que se repite en cada prueba de componente

POR QUE: TarjetaProducto usa Link, useCart y useTablasMaestras. Sin envolver con router y sin datos seed, el test falla aunque el componente este bueno.

```
import { MemoryRouter } from 'react-router-dom';
import { AuthProvider } from './context/AuthContext';
import { CartProvider } from './context/CartContext';
import { inicializarDatos } from './data/seed';
beforeEach(() => { localStorage.clear(); inicializarDatos(); });
```

```
function envolver(ui) {
  return (<MemoryRouter><AuthProvider><CartProvider>{ui}</CartProvider></AuthProvider></MemoryRouter>);
}
```

Guarda este patron: render(envolver(<TuComponente />)) en vez de render(<TuComponente />).

4. Prueba 1 - Renderizado correcto (lista)

TAREA PDF: componentes de lista rendericen todos los elementos.

POR QUE: demuestra que el map de Productos.tsx:63 no se come datos.

Archivo: src/pages/Productos.test.tsx (VS Code). Codigo:

```
import { describe, test, expect, beforeEach } from 'vitest';
import { render, screen } from '@testing-library/react';
import Productos from './Productos';
// + envolver de arriba
describe('Productos lista', () => {
  test('renderiza todas las tarjetas', () => {
    render(envolver(<Productos />));
    expect(screen.getAllByText(/Anadir al carrito/)).toHaveLength(17);
  });
});
```

Verificacion: npm test. Verde = OK. Si el numero no es 17, cambia el 17 por tu cantidad de seed (cuenta tus PRODUCTOS en seed.ts). Rojo getAllByText = no renderizo nada: revisa que inicializarDatos() corrio.

5. Prueba 2 - Renderizado condicional (mensaje solo con error)

TAREA PDF: mensaje de error solo cuando hay error.

POR QUE: prueba el if (carrito.length === 0) de Carrito.tsx:33-38.

Archivo: src/pages/Carrito.test.tsx

```
describe('Carrito vacio', () => {
  test('muestra mensaje cuando esta vacio', () => {
    render(envolver(<Carrito />));
    expect(screen.getByText(/Tu carrito esta vacio/)).toBeInTheDocument();
  });
});
```

Verificacion: pasa con carrito vacio (localStorage limpio). Si agregas un item, ese texto debe desaparecer: eso es lo condicional.

6. Prueba 3 - Props (boton recibe etiqueta y onClick)

TAREA PDF: boton recibe etiqueta y funcion onClick.

POR QUE: si pasas mal la prop producto, TarjetaProducto.tsx:35-36 no muestra el nombre. Asi pruebas props sin montar toda la tienda.

Archivo: src/components/common/TarjetaProducto.test.tsx

```
const fake = { codigo:'CH-001', nombre:'Aros Dorados', precio:11500, stock:15, stockCritico:4,
categoriaId:3, imagen:'x.jpg', estado:'Activo' };
describe('TarjetaProducto props', () => {
  test('muestra nombre y boton', () => {
    render(envolver(<TarjetaProducto producto={fake} />));
    expect(screen.getByText('Aros Dorados')).toBeInTheDocument();
    expect(screen.getByRole('button', { name: /Anadir al carrito/ })).toBeInTheDocument();
```

```
  });
});
```

Verificacion: si cambias fake.nombre y el test sigue buscando el viejo, se pone rojo. Eso demuestra que usa la prop.

7. Prueba 4 - Estado (formulario cambia al escribir)

TAREA PDF: estado del formulario cambia cuando el usuario escribe.

POR QUE: prueba el useState + filtrar() de Productos.tsx:12-17.

Archivo: agregar en Productos.test.tsx

```
import userEvent from '@testing-library/user-event';
test('escribir filtra', async () => {
  const user = userEvent.setup();
  render(envolver(<Productos />));
  const input = screen.getByPlaceholderText(/Buscar accesorio/);
  await user.type(input, 'aros');
  expect(input).toHaveValue('aros');
});
```

Verificacion: si el input no tiene value ligado al estado, toHaveValue falla. Recuerda await + setup, sin eso da falso rojo.

8. Prueba 5 - Eventos (clic cambia estado)

TAREA PDF: clic cambia estado o ejecuta funcion.

POR QUE: prueba el onClick de TarjetaProducto.tsx:47 que llama agregar().

Archivo: agregar en TarjetaProducto.test.tsx

```
test('clic agrega al carrito', async () => {
  const user = userEvent.setup();
  render(envolver(<TarjetaProducto producto={fake} />));
  await user.click(screen.getByRole('button', { name: /Anadir al carrito/ }));
  expect(JSON.parse(localStorage.getItem('chic_carrito'))).toHaveLength(1);
});
```

Verificacion: revisa tu clave real en claves.ts (chic_carrito). Si tu clave es otra, cambia el string. Rojo aqui = el onClick no guarda.

9. Como entregar cobertura

- Corre npm test y guarda captura en verde.
- Corre npm test -- --coverage para generar coverage/.
- En tu documento haz una tabla por cada prueba 1-5: que verifica, archivo, dato entrada, salida esperada, resultado.
- Errores tipicos: pegar codigo en PowerShell (va en VS Code), olvidar envolver con MemoryRouter (error useNavigate), olvidar inicializarDatos (lista vacia), clave localStorage distinta.